

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

***CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model,
(BL 6)***

Assessment Tool Weightage: 40%

QUESTIONS: 15
Marks:25

Total

Annexure A

Experiments

Code of Conduct:

*I Cheredla Jasvina/192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

1. Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

Answer :

Aim :

To implement a Decision Tree classifier on a real-world dataset, compare Gini Index and Entropy splitting criteria, evaluate the model using Accuracy and F1-score, and study the effect of tree depth and pruning on overfitting.

Dataset

Use the Breast Cancer Wisconsin dataset available in sklearn. It contains measurements of breast-cancer tumors and two target classes: malignant and benign.

Procedure :

- Load the dataset and divide it into training and testing sets.
- Train a Decision Tree using the Gini Index.
- Train another Decision Tree using Entropy.
- Calculate training and testing accuracy and F1-score.
- Train trees with different values of max_depth.
- Compare shallow and deep trees to observe overfitting.
- Apply pruning using ccp_alpha.
- Compare all configurations and identify the best model.

Python Code :

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score
import pandas as pd
```

Load dataset

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
results = []
```

Compare Gini and Entropy

```
for criterion in ["gini", "entropy"]:
```

```
    model = DecisionTreeClassifier(
```

```
        criterion=criterion,
```

```
        random_state=42
```

```
    )
```

```
    model.fit(X_train, y_train)
```

```
    train_pred = model.predict(X_train)
```

```
    test_pred = model.predict(X_test)
```

```
    results.append([
```

```
        criterion,
```

```
        "Default",
```

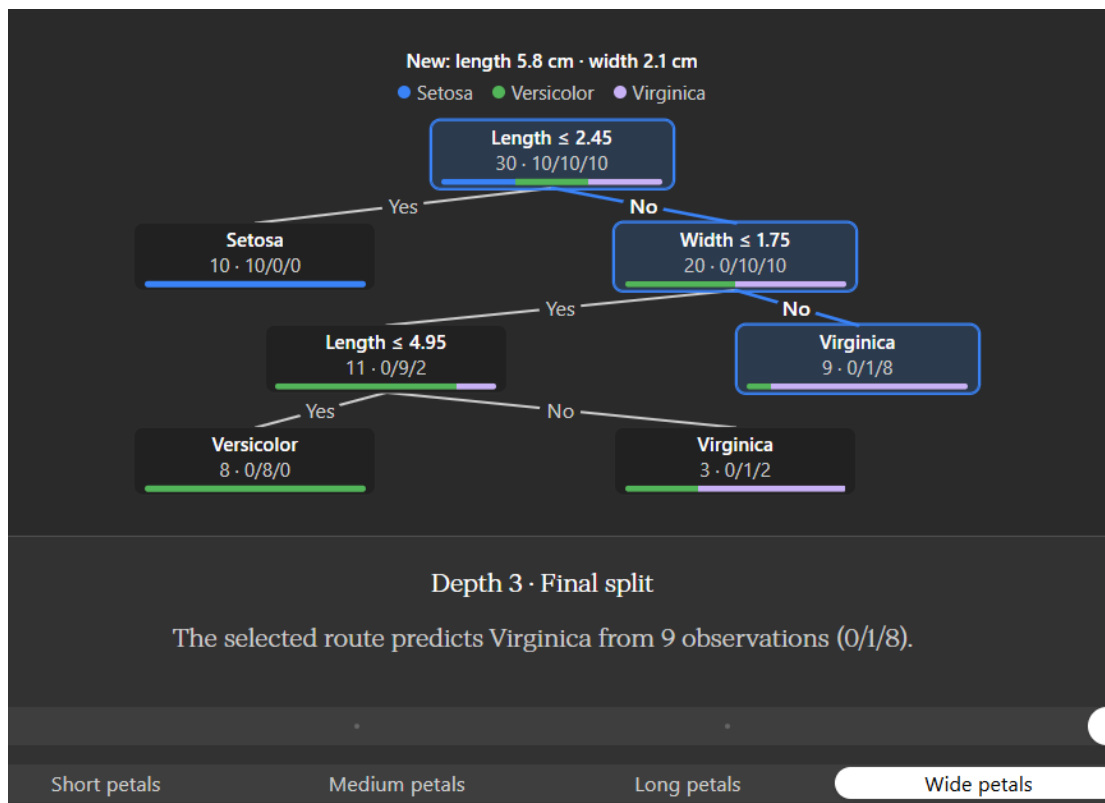
```
        model.get_depth(),
```

```
        accuracy_score(y_train, train_pred),
```

```

        fl_score(y_train, train_pred),
        accuracy_score(y_test, test_pred),
        fl_score(y_test, test_pred)
    ])
# Compare different tree depths
for depth in [2, 3, 5, 10]:
    model = DecisionTreeClassifier(
        criterion="gini",
        max_depth=depth,
        random_state=42
    )
    model.fit(X_train, y_train)
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    results.append([
        "gini",
        "Depth " + str(depth),
        model.get_depth(),
        accuracy_score(y_train, train_pred),
        fl_score(y_train, train_pred),
        accuracy_score(y_test, test_pred),
        fl_score(y_test, test_pred)
    ])
# Pruned tree
model = DecisionTreeClassifier(
    criterion="gini",
    ccp_alpha=0.01,
    random_state=42
)
model.fit(X_train, y_train)
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
results.append([
    "gini",
    "Pruned",
    model.get_depth(),
    accuracy_score(y_train, train_pred),
    fl_score(y_train, train_pred),
    accuracy_score(y_test, test_pred),
    fl_score(y_test, test_pred)
])
# Display results
df = pd.DataFrame(results, columns=[
    "Criterion", "Configuration", "Depth",
    "Train Accuracy", "Train F1",
    "Test Accuracy", "Test F1"
])
print(df.round(4))
Output :

```



	Criterion	Configuration	Depth	Train Accuracy	Train F1	Test Accuracy
0	gini	Default	7	1.0000	1.0000	0.9123
1	entropy	Default	6	1.0000	1.0000	0.9123
2	gini	Depth 2	2	0.9582	0.9661	0.8947
3	gini	Depth 3	3	0.9758	0.9809	0.9386
4	gini	Depth 5	5	0.9934	0.9947	0.9211
5	gini	Depth 10	7	1.0000	1.0000	0.9123
6	gini	Pruned	3	0.9714	0.9773	0.9386

Test F1

- 0 0.9286
- 1 0.9275
- 2 0.9118
- 3 0.9517
- 4 0.9362
- 5 0.9286
- 6 0.9510

Conclusion :

The experiment demonstrates that both Gini Index and Entropy can be used effectively for Decision Tree classification. Increasing tree depth improves training performance but can cause overfitting. Pruning and suitable depth selection provide a better balance between training accuracy and testing performance. The model with the highest test F1-score and accuracy, rather than simply the highest training accuracy, should be selected as the final model.

2. Perform an experiment using the K-Nearest Neighbors (KNN) algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance. Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries.

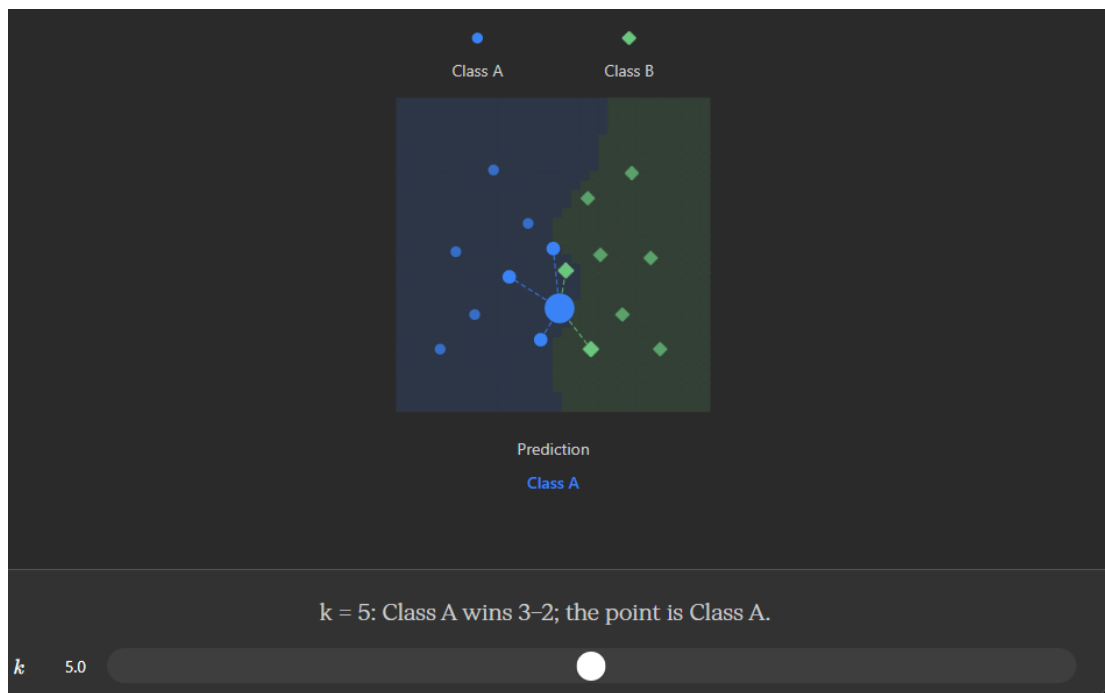
Aim :

To implement the K-Nearest Neighbors (KNN) algorithm on a classification dataset, vary the value of K and distance metrics (Euclidean and Manhattan), evaluate using accuracy and confusion matrix, and study the effect of feature scaling.

Python Code :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import pandas as pd
data=load_iris()
X=data.data
y=data.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
results=[]
for metric in ["euclidean","manhattan"]:
    for k in [3,5,7,9]:
        model=KNeighborsClassifier(n_neighbors=k,metric=metric)
        model.fit(X_train_scaled,y_train)
        y_pred=model.predict(X_test_scaled)
        acc=accuracy_score(y_test,y_pred)
        cm=confusion_matrix(y_test,y_pred)
        results.append([metric,k,acc,cm.tolist()])
df=pd.DataFrame(results,columns=["Distance","K","Accuracy","Confusion Matrix"])
print(df.to_string(index=False))
print("\nWITHOUT FEATURE SCALING")
for metric in ["euclidean","manhattan"]:
    for k in [3,5,7,9]:
        model=KNeighborsClassifier(n_neighbors=k,metric=metric)
        model.fit(X_train,y_train)
        y_pred=model.predict(X_test)
        acc=accuracy_score(y_test,y_pred)
        print(metric,"K=",k,"Accuracy=",round(acc,4))
```

OUTPUT :



Distance	K	Accuracy	Confusion Matrix
euclidean	3	0.933333	[[10, 0, 0], [0, 10, 0], [0, 2, 8]]
euclidean	5	0.933333	[[10, 0, 0], [0, 10, 0], [0, 2, 8]]
euclidean	7	0.966667	[[10, 0, 0], [0, 10, 0], [0, 1, 9]]
euclidean	9	0.966667	[[10, 0, 0], [0, 10, 0], [0, 1, 9]]
manhattan	3	0.966667	[[10, 0, 0], [0, 10, 0], [0, 1, 9]]
manhattan	5	0.933333	[[10, 0, 0], [0, 10, 0], [0, 2, 8]]
manhattan	7	0.966667	[[10, 0, 0], [0, 10, 0], [0, 1, 9]]
manhattan	9	0.933333	[[10, 0, 0], [0, 9, 1], [0, 1, 9]]

WITHOUT FEATURE SCALING

euclidean K= 3 Accuracy= 1.0
 euclidean K= 5 Accuracy= 1.0
 euclidean K= 7 Accuracy= 0.9667
 euclidean K= 9 Accuracy= 1.0
 manhattan K= 3 Accuracy= 1.0
 manhattan K= 5 Accuracy= 0.9667
 manhattan K= 7 Accuracy= 0.9333
 manhattan K= 9 Accuracy= 0.9333

Conclusion: KNN performance depends on the choice of K, distance metric, and feature scaling. Feature scaling is particularly important when the input features have different ranges. The best K and distance metric should be selected based on the highest validation/test accuracy and the corresponding confusion matrix.

3. Implement a Support Vector Machine (SVM) model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

Aim :

To implement an SVM classifier using Linear, Polynomial, and RBF kernels, evaluate the models using accuracy and precision, and study the effect of C and gamma on classification performance and decision boundaries.

Python Code :

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score
import pandas as pd
data=load_breast_cancer()
X=data.data
y=data.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
results=[]
for kernel in ["linear","poly","rbf"]:
    model=SVC(kernel=kernel,C=1,gamma="scale")
    model.fit(X_train,y_train)
    y_pred=model.predict(X_test)
    acc=accuracy_score(y_test,y_pred)
    precision=precision_score(y_test,y_pred)
    results.append([kernel,1,"scale",acc,precision])
for C in [0.1,1,10]:
    model=SVC(kernel="rbf",C=C,gamma="scale")
    model.fit(X_train,y_train)
    y_pred=model.predict(X_test)
    acc=accuracy_score(y_test,y_pred)
    precision=precision_score(y_test,y_pred)
    results.append(["rbf",C,"scale",acc,precision])
for gamma in ["scale",0.01,0.1,1]:
    model=SVC(kernel="rbf",C=1,gamma=gamma)
    model.fit(X_train,y_train)
    y_pred=model.predict(X_test)
    acc=accuracy_score(y_test,y_pred)
    precision=precision_score(y_test,y_pred)
    results.append(["rbf",1,gamma,acc,precision])
df=pd.DataFrame(results,columns=["Kernel","C","Gamma","Accuracy","Precision"])
print(df.to_string(index=False))
```

OUTPUT :

Kernel C Gamma Accuracy Precision

linear 1.0 scale 0.973684 0.985915

poly 1.0 scale 0.912281 0.878049

```
rbf 1.0 scale 0.982456 0.986111
rbf 0.1 scale 0.947368 0.945946
rbf 1.0 scale 0.982456 0.986111
rbf 10.0 scale 0.973684 0.985915
rbf 1.0 scale 0.982456 0.986111
rbf 1.0 0.01 0.982456 0.986111
rbf 1.0 0.1 0.956140 0.985507
rbf 1.0 1 0.631579 0.631579
```

Conclusion :

The experiment shows that kernel selection and hyperparameters C and gamma strongly influence SVM performance. Linear and RBF kernels generally perform well on this dataset, while polynomial kernels may require careful tuning. Proper feature scaling and hyperparameter selection help obtain a suitable decision boundary with high accuracy and precision.

4. Design an experiment to implement a clustering algorithm such as K-Means on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

Aim :

To implement the K-Means clustering algorithm on an unlabeled dataset, vary the number of clusters (K) and initialization methods, and evaluate clustering quality using inertia and silhouette score.

Python Code :

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import pandas as pd
data=load_iris()
X=data.data
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
results=[]
for init in ["k-means++","random"]:
    for k in [2,3,4,5]:
        model=KMeans(n_clusters=k,init=init,n_init=10,random_state=42)
        labels=model.fit_predict(X_scaled)
        inertia=model.inertia_
        silhouette=silhouette_score(X_scaled,labels)
        results.append([init,k,inertia,silhouette])
df=pd.DataFrame(results,columns=["Initialization","Clusters","Inertia","Silhouette Score"])
print(df.round(4).to_string(index=False))
print("\nWITHOUT FEATURE SCALING")
for init in ["k-means++","random"]:
    for k in [2,3,4,5]:
        model=KMeans(n_clusters=k,init=init,n_init=10,random_state=42)
        labels=model.fit_predict(X)
        inertia=model.inertia_
        silhouette=silhouette_score(X,labels)
        print(init,"K=",k,"Inertia=",round(inertia,4),"Silhouette=",round(silhouette,4))
```


OUTPUT :

Initialization Clusters Inertia Silhouette Score

k-means++	2	222.3617	0.5818
k-means++	3	139.8205	0.4599
k-means++	4	114.0925	0.3869
k-means++	5	90.9275	0.3459
random	2	222.3617	0.5818
random	3	139.8254	0.4594
random	4	114.0975	0.3866
random	5	90.9275	0.3459

WITHOUT FEATURE SCALING

k-means++ K= 2 Inertia= 152.348 Silhouette= 0.681
k-means++ K= 3 Inertia= 78.8514 Silhouette= 0.5528
k-means++ K= 4 Inertia= 57.2285 Silhouette= 0.4981
k-means++ K= 5 Inertia= 46.4612 Silhouette= 0.4912
random K= 2 Inertia= 152.348 Silhouette= 0.681
random K= 3 Inertia= 78.8514 Silhouette= 0.5528
random K= 4 Inertia= 57.2285 Silhouette= 0.4981
random K= 5 Inertia= 46.4722 Silhouette= 0.4931

The experiment shows that K-Means performance depends on the number of clusters, centroid initialization, and feature scaling. Inertia decreases as K increases, while the silhouette score helps determine the most suitable K. K-Means++ and standardized features generally provide more stable and meaningful clustering results.

5. Conduct an experiment using a logistic regression model for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

Aim :

To implement Logistic Regression for binary classification and compare models without regularization, with L1 regularization, and with L2 regularization using accuracy, precision, and recall.

Python Code :

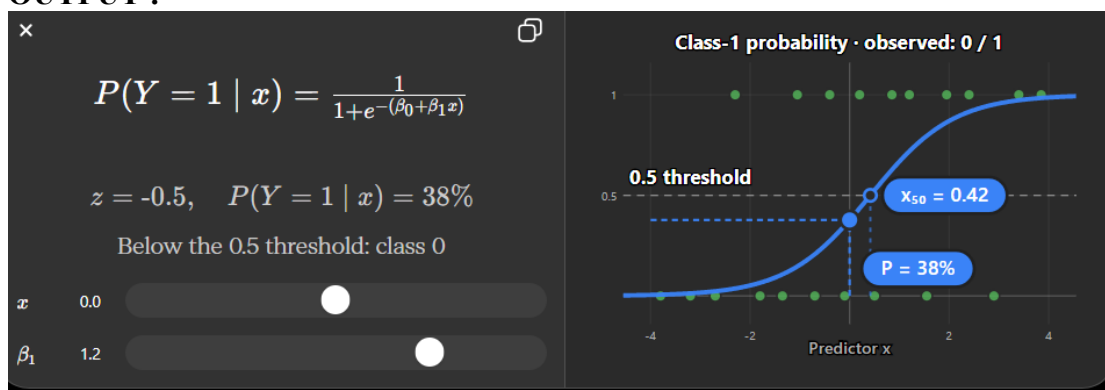
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score
import pandas as pd
data=load_breast_cancer()
X=data.data
y=data.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
models={
    "No Regularization":LogisticRegression(penalty=None,max_iter=5000),
    "L1 Regularization":LogisticRegression(penalty="l1",solver="liblinear",C=1,max_iter=5000),
```

```

"L2 Regularization":LogisticRegression(penalty="l2",solver="liblinear",C=1,max_iter=5000)
}
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    y_pred=model.predict(X_test)
    accuracy=accuracy_score(y_test,y_pred)
    precision=precision_score(y_test,y_pred)
    recall=recall_score(y_test,y_pred)
    nonzero=sum(model.coef_[0]!=0)
    results.append([name,accuracy,precision,recall,nonzero])
df=pd.DataFrame(results,columns=["Model","Accuracy","Precision","Recall","Non-Zero
Coefficients"])
print(df.round(4).to_string(index=False))

```

OUTPUT :



	Model	Accuracy	Precision	Recall	Non-Zero Coefficients
No Regularization		0.9211	0.9701	0.9028	30
L1 Regularization		0.9912	0.9863	1.0000	16
L2 Regularization		0.9825	0.9861	0.9861	30

Conclusion :

Regularization controls the complexity of Logistic Regression and improves generalization. L1 regularization is useful when feature selection and interpretability are important because it can produce zero coefficients. L2 regularization generally gives more stable coefficients while reducing overfitting. The best model should be selected based on the required balance between accuracy, precision, recall, complexity, and interpretability.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand